

CURSO PRÁCTICO .NET 8 / EF CORE

Proyecto básico Entity Framework Core

Creación paso a paso de una aplicación web de gestión de tareas con Razor
Pages y SQLite

Introducción al Proyecto

Modelo de Datos Simplificado

Construiremos una aplicación web centrada en un único modelo de dominio principal: la entidad **Tarea**.

- ✓ **Descripción:** Cadena de texto con el detalle de la labor a realizar.
- ✓ **Estado:** Valor booleano para indicar si está completada o pendiente.
- ✓ **Fecha:** Timestamp de registro automático en el sistema.

Alcance Funcional (CRUD)

El objetivo pedagógico es dominar las operaciones fundamentales de persistencia de datos:

Operaciones Básicas

Create (Crear tarea) • Read (Listar tareas) • Update (Cambiar estado completado/pendiente) • Delete (Eliminar tarea).

> Creación del Proyecto

Comando .NET CLI

```
$ dotnet new razor -n Tareas
```

Ejecuta este comando en tu terminal para andamiar una solución basada en ASP.NET Core Razor Pages.

Concepto: Arquitectura Razor Pages

Enfoque Orientado a Páginas

Razor Pages organiza el código alrededor de capacidades de UI específicas (archivos .cshtml y sus modelos correspondientes .cshtml.cs), facilitando un desarrollo más limpio y encapsulado que el MVC convencional para aplicaciones basadas en formularios.

Instalación de Dependencias

Paquetes Requeridos

-  **Provider SQLite:** Permite traducir las consultas LINQ a sentencias SQL para una base de datos ligera integrada.
-  **EF Core Tools:** Proporciona soporte en tiempo de diseño para la ejecución de migraciones y actualización del esquema.

Comandos de Instalación

```
$ dotnet add package Microsoft.EntityFrameworkCore.Sqlite  
  
$ dotnet add package Microsoft.EntityFrameworkCore.Design
```

Modelo de Dominio: Tarea

Conceptos Clave de la Entidad

Identificación Automática

EF Core reconoce la propiedad nombrada Id como la Clave Primaria (PK) autonumérica de la tabla de forma implícita.

Valores por Defecto

Inicializar FechaCreacion con DateTime.Now garantiza la integridad del registro temporal desde la capa de aplicación.

Models/Tarea.csC#

```
namespace Tareas.Models;

public class Tarea
{
    public int Id { get; set; }
    public string Descripcion { get; set; } = "";
    public bool Completada { get; set; }
    public DateTime FechaCreacion { get; set; } = DateTime.Now;
}
```

Contexto de Datos ApplicationDbContext

Conceptos del DbContext

Constructores Primarios

Usamos la sintaxis moderna de C# 12 declarando los parámetros directamente en la clase:
AppDbContext(options) reduciendo código repetitivo.

Mapeo con DbSet

DbSet representa la tabla lógica en la base de datos y sirve como punto de entrada para las consultas LINQ.

Data/AppDbContext.cs# 12

```
using Microsoft.EntityFrameworkCore;
using Tareas.Models;

namespace Tareas.Data; // Uso de Constructor Primario para heredar DbContext
public class AppDbContext(DbContextOptions<AppDbContext> options) :
    DbContext(options)
{
    public DbSet<Tarea> Tareas => Set<Tarea>();
}
```

Interfaz Servicio ITareasService

Abstracción y Contrato

Principio de Inversión de Dependencias

Definir la interfaz independiza las páginas UI de la implementación técnica concreta de acceso a datos.

Diseño Evolutivo

Iniciamos con un diseño sincrónico directo para comprender el flujo de negocio antes de introducir complejidad asíncrona.

Services/ITareasService.csC#

```
using Tareas.Models;
namespace Tareas.Services;

public interface ITareasService
{
    List<Tarea> GetTodasLasTareas();
    void CrearTarea(string descripcion);
    void ToggleEstadoTarea(int id);
    void EliminarTarea(int id);
}
```

☰ Servicio: Obteniendo Tareas

</> Inyección y Consulta LINQ

Inyección con Constructor Primario

TareasService(AppDbContext context) inyecta el contexto de datos de forma concisa sin campos privados explícitos.

Ejecución Inmediata

El método ToList() ejecuta la consulta SQL en la base de datos y materializa los registros en memoria.

Services/TareasService.cs (Parte 1)C# 12

```
using Tareas.Data;
using Tareas.Models;
namespace Tareas.Services;

public class TareasService(AppDbContext context) : ITareasService
{
    public List<Tarea> GetTodasLasTareas() {
        return context.Tareas.ToList();
    }
}
```


⚡ Operaciones Directas (ExecuteUpdate/Delete)

🕒 Rendimiento sin Change Tracker

Evitar la Carga de Entidades

En lugar del patrón tradicional de 2 pasos (hacer un Find() y luego SaveChanges()), los métodos modernos ExecuteUpdate y ExecuteDelete ejecutan directamente sentencias SQL UPDATE o DELETE en la BD.

Única Consulta en BBDD

Se elimina una ida y vuelta (roundtrip) a la base de datos y se evita registrar objetos en el Change Tracker cuando no es necesario.

Services/TareasService.cs (Parte 2) C# Moderno EF Core 7+

```
public void CrearTarea(string descripcion)
{
    context.Tareas.Add(new Tarea { Descripcion = descripcion });
    context.SaveChanges();
}

public void ToggleEstadoTarea(int id)
{
    // Un solo UPDATE SQL directo en la base de datos
    context.Tareas.Where(t => t.Id == id)
        .ExecuteUpdate(s => s.SetProperty(t => t.Completada, t => !t.Completada));
}

public void EliminarTarea(int id)
{
    // Un solo DELETE SQL directo en la base de datos
    context.Tareas.Where(t => t.Id == id).ExecuteDelete();
}
```

Configuración en Program.cs

Registro en el Contenedor DI

Inyección de DbContext

AddDbContext registra la BD usando el proveedor SQLite apuntando a la cadena de conexión definida.

Vida Útil Scoped

AddScoped asegura que se cree una instancia del servicio por cada solicitud HTTP recibida, compartiendo el mismo contexto de datos.

Program.csC#

```
using Microsoft.EntityFrameworkCore;
using Tareas.Data;
using Tareas.Services;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddRazorPages(); // Configuración de SQLite
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlite("Data Source=tareas.db")); // Registro de Servicios
builder.Services.AddScoped<ITareasService, TareasService>(); var app =
builder.Build(); app.MapRazorPages(); app.Run();
```

Ventajas de IReadOnlyList



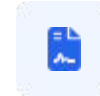
Inmutabilidad de la Colección

Garantiza que las capas externas (como la UI) puedan leer los datos pero no modificar la estructura de la lista (evita operaciones como Add o Remove directas).



Encapsulamiento del Dominio

Protege el estado interno del servicio, obligando a que cualquier alteración de datos pase explícitamente por los métodos del servicio diseñados para tal fin.



Contrato de API Claro

Comunica de manera explícita la intención del método a otros desarrolladores: obtener una vista de datos únicamente para lectura.

IndexModel: Vista y Listado

Pages/Index.cshtml.cs# 12

```
public class IndexModel(ITareasService service) : PageModel
{
    public IReadOnlyList<Tarea> Tareas { get; set; } = [];
    public void OnGet() { Tareas = service.GetTodasLasTareas(); }
}
```

Pages/Index.cshtmlHTML / Razor

```
<h1>Lista de Tareas</h1>
<ul>
    @foreach (var item in Model.Tareas)
    {
        <li>
            @item.Descripcion -
            @(item.Completada ? "Completada" : "Pendiente")
        </li>
    }
</ul>
```

⚡ IndexModel: Acciones OnPost

➤ Manejadores de Eventos POST

Manejadores Específicos (Named Handlers)

Razor Pages permite declarar métodos como OnPostToggle que responden directamente a formularios con el atributo asp-page-handler="Toggle".

Patrón Post-Redirect-Get (PRG)

Devolver RedirectToPage() tras procesar una acción evita el reenvío duplicado del formulario al recargar la página.

Pages/Index.cshtml.csC#

```
public IActionResult OnPostToggle(int id)
{
    service.ToggleEstadoTarea(id);
    return RedirectToPage();
}

public IActionResult OnPostDelete(int id)
{
    service.EliminarTarea(id);
    return RedirectToPage();
}
```

+ CreateModel: Formulario e Input

Modelo de Entrada Con Records

Uso de Records para Entrada

CreateTareaInput como record inmutable encapsula los datos requeridos únicamente para la validación del formulario de creación.

Atributo [BindProperty]

Enlaza automáticamente los datos enviados en el POST HTTP directamente a las propiedades del modelo de la página.

Pages/Create.cshtml.cs# 12

```
namespace Tareas.Pages;

// Record de entrada para el formulario
public record CreateTareaInput([Required] string Descripcion);

public class CreateModel(ITareasService service) : PageModel
{
    [BindProperty]
    public CreateTareaInput Input { get; set; } = default!;

    public void OnGet() { }
}
```

CreateModel: Guardado de Datos

✓ Flujo de Validación y Creación

Verificación de ModelState

ModelState.IsValid valida las anotaciones de datos (como [Required]) antes de invocar la lógica de negocio.

Navegación Controlada

Si la entrada es inválida, se re-renderiza la página actual mostrando los mensajes de error; si es válida, redirige al listado principal.

Pages/Create.cshtml.csC#

```
public IActionResult OnPost()
{
    // Validar errores de binding
    if (!ModelState.IsValid)
    {
        return Page();
    }

    // Procesar la creación de la tarea
    service.CrearTarea(Input.Descripcion);

    return RedirectToPage("Index");
}
```

↶ Patrón DTO y Justificación

🛡️ ¿Por qué desacoplar con DTOs?

Desacoplamiento UI de la BD

Evita exponer directamente el modelo ORM (Tarea) a la capa de vista, protegiendo la estructura interna de la base de datos.

Prevención de Over-Posting

Impide que usuarios malintencionados modifiquen campos sensibles enviando parámetros no deseados en la petición HTTP.

DT0s/TareaDT0.cs# 12 Record

```
namespace Tareas.DTOs;

// DTO immutable para transferencia de datos
public record TareaDT0(
    int Id,
    string Descripcion,
    bool Completada,
    DateTime FechaCreacion
);
```


> Comandos con C# Records

🎯 Intención Clara y CQRS

Representación Explícita de Acciones

En lugar de pasar parámetros sueltos, cada comando encapsula formalmente la intención del usuario para modificar el sistema.

Igualdad por Valor

Los records de C# proveen inmutabilidad y comparación de valores por defecto, ideales para capas de aplicación.

Commands/TareasCommands.csC# 12

```
namespace Tareas.Commands;

// Comando para creación de tarea
public record CreateTareaCommand(string Descripcion);

// Comando para actualización de estado
public record UpdateTareaStateCommand(int Id, bool Completada);
```

Asincronía y CancellationToken



Liberación de Hilos

Las operaciones I/O de base de datos liberan los hilos del servidor web (Thread Pool) mientras esperan la respuesta de disco o red, aumentando drásticamente la escalabilidad.



Cancelación Pronta

Un CancellationToken propaga la cancelación cuando el usuario cierra la pestaña o navega fuera, abortando consultas pesadas en la BD.



Saturación Evitada

Sostener operaciones síncronas bloqueantes en servidores bajo alta carga produce agotamiento de hilos (Thread Starvation).

Servicio Asíncrono EF Core

Transformación a Async Directo

Operaciones Bulk Asíncronas

Usamos `ExecuteUpdateAsync` y `ExecuteDeleteAsync` para modificar o borrar registros en una sola sentencia SQL asíncrona.

Propagación de `CancellationToken`

Pasamos el token `ct` a `ToListAsync()`, `ExecuteUpdateAsync()` y `ExecuteDeleteAsync()` para cancelar la consulta si se aborta el Request HTTP.

Services/TareasService.cs# Async EF Core

```
public async Task<IReadOnlyList<Tarea>> GetTodasLasTareasAsync(
    CancellationToken ct = default)
{
    return await context.Tareas.ToListAsync(ct);
}

public async Task ToggleEstadoTareaAsync(
    int id, CancellationToken ct = default)
{
    await context.Tareas
        .Where(t => t.Id == id)
        .ExecuteUpdateAsync(s => s.SetProperty(
            t => t.Completada, t => !t.Completada), ct);
}

public async Task EliminarTareaAsync(
    int id, CancellationToken ct = default)
{
    await context.Tareas
        .Where(t => t.Id == id)
        .ExecuteDeleteAsync(ct);
}
```

Refactor de IndexModel a Async

Convención de Nombres en Razor

Sufijo Async Obligatorio

ASP.NET Core requiere que los manejadores asíncronos utilicen el sufijo Async (ej. OnGetAsync, OnPostToggleAsync).

Inyección del Token de Cancelación

ASP.NET Core inyecta automáticamente el token de la solicitud HTTP actual directamente en los parámetros del manejador.

Pages/Index.cshtml.cs# Async

```
public async Task OnGetAsync(CancellationTokens ct)
{
    Tareas = await service.GetTodasLasTareasAsync(ct);
}

public async Task<IActionResult> OnPostToggleAsync(
    int id, CancellationTokens ct)
{
    await service.ToggleEstadoTareaAsync(id, ct);
    return RedirectToPage();
}
```

✓ Refactor de CreateModel a Async

Flujo Asíncrono End-to-End

Sin Bloqueos de Extremo a Extremo

Desde la petición HTTP recibida en Razor Pages hasta el driver SQLite de bajo nivel, la ejecución no bloquea ningún hilo de la aplicación.

Código Moderno y Robusto

Se combinan Constructores Primarios, Records, validación estructurada y asincronía total siguiendo los estándares modernos de .NET.

Pages/Create.cshtml.csC# Async

```
public async Task<IActionResult> OnPostAsync(
    CancellationToken cancellationToken)
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    await service.CrearTareaAsync(
        Input.Descripcion, cancellationToken);

    return RedirectToPage("Index");
}
```